KT AIVLE School Mini Project 5

클라우드 네이티브 웹 서비스 구축

CI/CD부터 운영 자동화

AWS EKS, CodePipeline, Auto Scaling을 활용한 고가용성 아키텍처 구현

김경훈

프로젝트 로드맵

Phase1: 초기설정 & CI

GitHub 연동 및 이미지 빌드 자동화 설정

CodePipeline, CodeBuild

→

Phase2: Deployment, Automation

EKS 배포 파이프라인 완성

서비스 수행 (CodeDeploy)

→

Phase3: Resilience

트래픽 대응을 위한 오토 스케일링

모니터링 구축(ALB, HPA, CloudWatch)

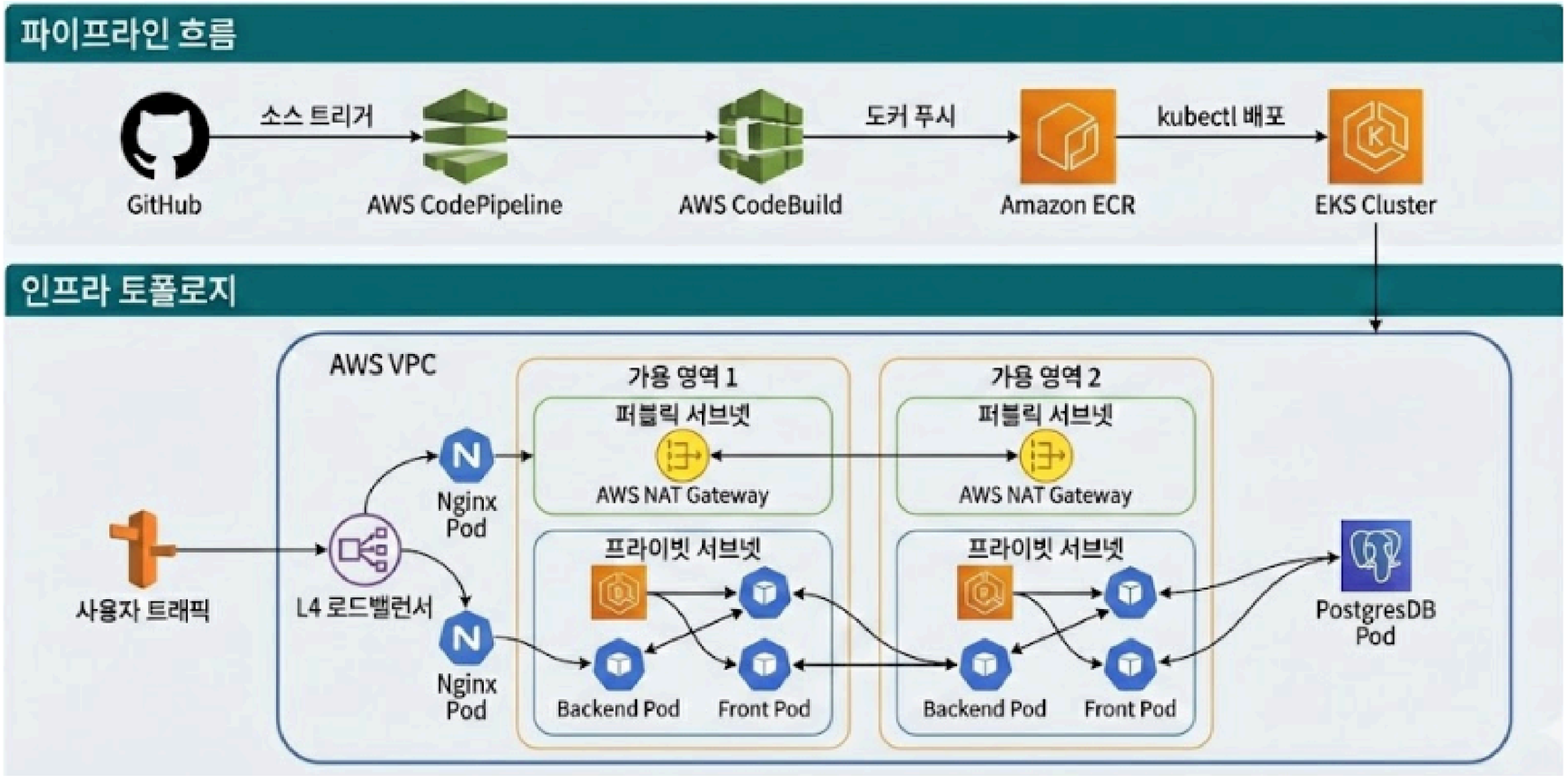
→

Phase4: Governace

운영 안정성을 위한 수동 승인 추가

알림 체계(Manual Approval)

시스템 아키텍처



Seamless Integration

GitHub-CodePipeline 기반의 완전 자동화된 빌드/배포.

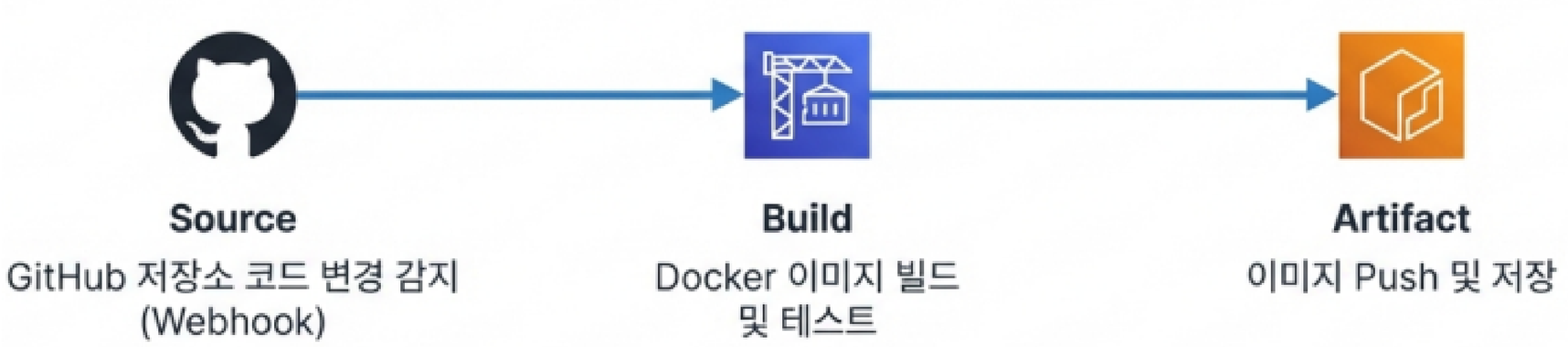
High Availability

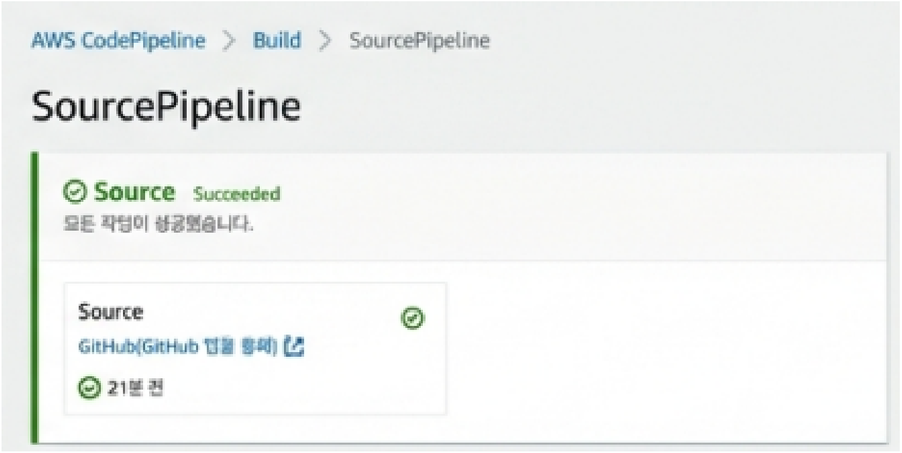
Multi-AZ 구조 및 HPA(Horizontal Pod Autoscaler) 적용.

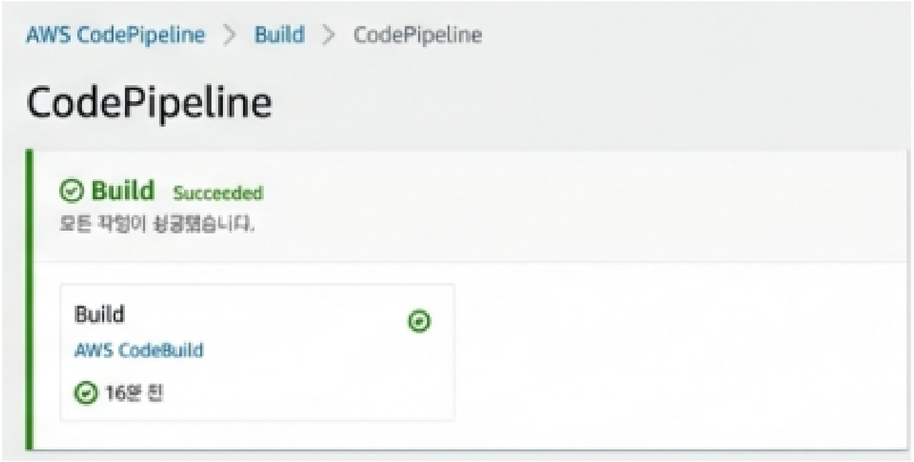
Observability

CloudWatch Container Insights를 통한 실시간 모니터링.

CI 파이프라인 구축


Source
GitHub 저장소 코드 변경 감지
(Webhook)
Build
Docker 이미지 빌드
및 테스트
Artifact
이미지 Push 및 저장


AWS CodePipeline > Build > SourcePipeline
SourcePipeline
Source Succeeded
모든 작업이 성공했습니다.
Source
GitHub(GitHub 앱을 통해)
21분 전


AWS CodePipeline > Build > CodePipeline
CodePipeline
Build Succeeded
모든 작업이 성공했습니다.
Build
AWS CodeBuild
16분 전

CI 파이프라인 구축

Pre_build: ECR 로그인
및 환경 변수 설정

Build: Gradle 빌드와 Docker 이미지 생성

Post_build: ECR Push 및 EKS 배포

```yaml
pre_build:
  commands:
    - echo "ECR Login"
    - aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-
    - REPO_BACKEND=416225317325.dkr.ecr.us-east-1.amazonaws.com/ai0102-backend
    - REPO_FRONTEND=416225317325.dkr.ecr.us-east-1.amazonaws.com/ai0102-front
    - COMMIT_HASH=$(echo $CODEBUILD_RESOLVED_SOURCE_VERSION | cut -c 1-7)
    - IMAGE_TAG=${COMMIT_HASH:=latest}

build:
  commands:
    - echo "Build Backend"
    - cd book
    - ./gradlew clean build -x test
    - docker build -t $REPO_BACKEND:latest .
    - docker tag $REPO_BACKEND:latest $REPO_BACKEND:$IMAGE_TAG
    - cd ..

    - echo "Build Frontend"
    - cd book/frontend
    - docker build -t $REPO_FRONTEND:latest .
    - docker tag $REPO_FRONTEND:latest $REPO_FRONTEND:$IMAGE_TAG
    - cd ../..

post_build:
  commands:
    - echo "Push images to ECR"
    - docker push $REPO_BACKEND:latest
    - docker push $REPO_BACKEND:$IMAGE_TAG
    - docker push $REPO_FRONTEND:latest
    - docker push $REPO_FRONTEND:$IMAGE_TAG

    - echo "Deploy to EKS"
    - aws eks update-kubeconfig --name ai0102-EKS --region us-east-1
    - kubectl apply -f k8s-deployment.yaml
    - echo "Deployment Complete"
```

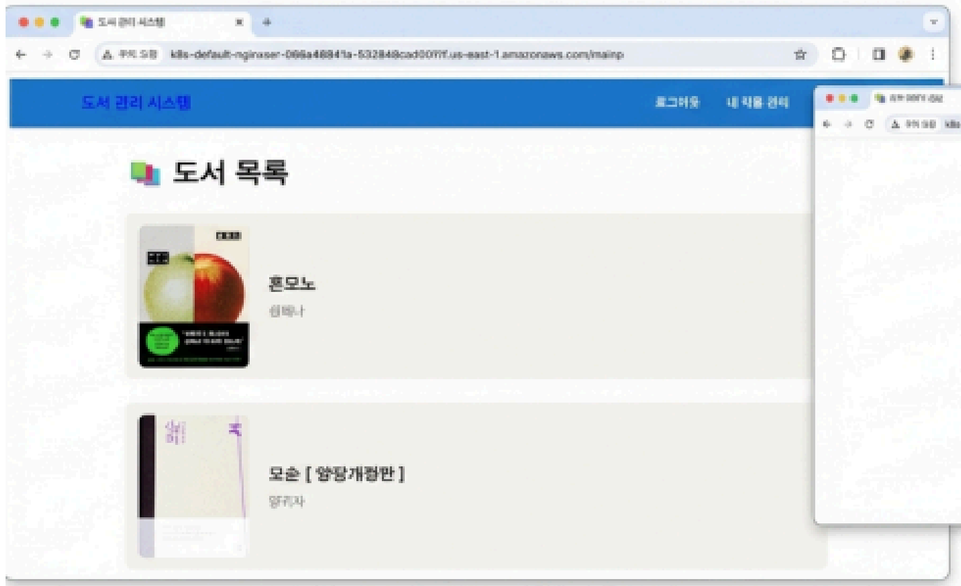
“표준화 빌드 스크립트를 통해 배포 일관성 확보“

CD 및 서비스 배포

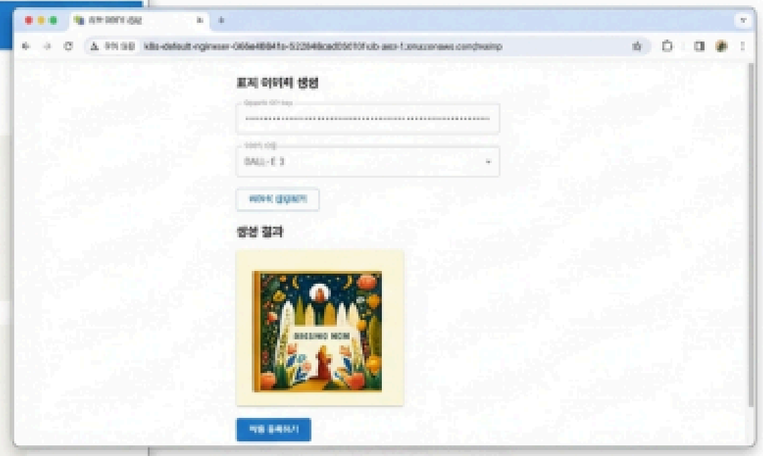
Agent Status

```
[ec2-user@ip-172-31-67-86 ~]$ sudo service codedeploy-agent status
The AWS CodeDeploy agent is running as PID 29236
```

Instance Health



Main Dashboard



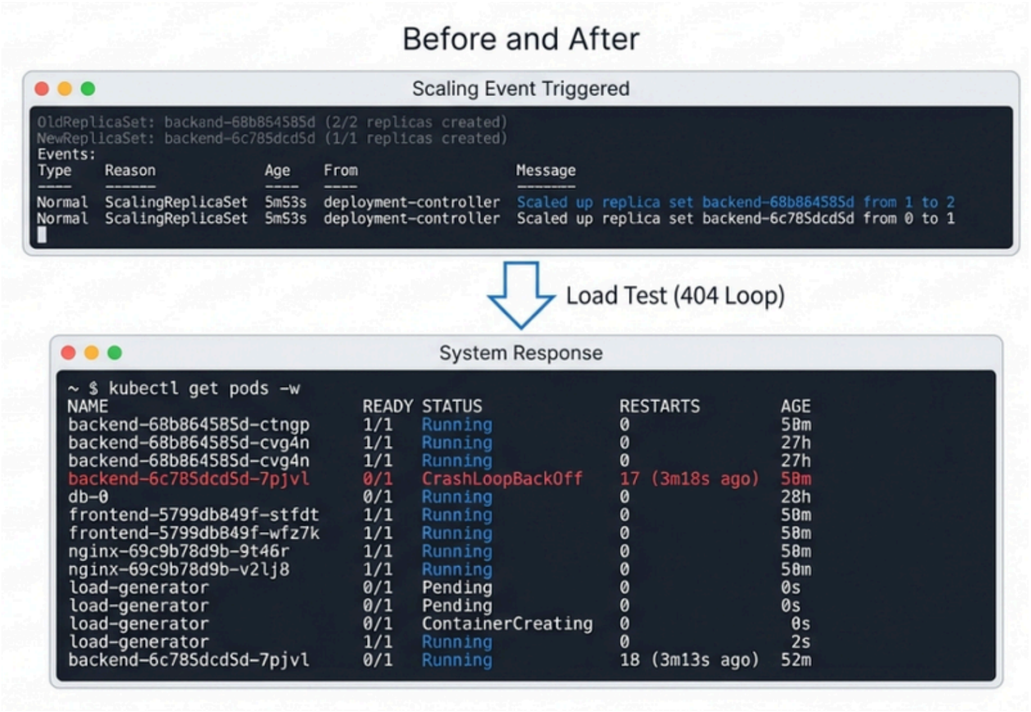
AI Feature Integration

HPA 및 AutoScaler 검증

```
~ $ kubectl get hpa -w
NAME      REFERENCE            TARGETS            MINPODS  MAXPODS  REPLICAS  AGE
backend   Deployment/backend   cpu: <unknown>/50%  2        5        2         49s
backend   Deployment/backend   cpu: <unknown>/50%  2        5        2         75s
backend   Deployment/backend   cpu: <unknown>/50%  2        5        2         90s
backend   Deployment/backend   cpu: <unknown>/50%  2        5        2         105s
backend   Deployment/backend   cpu: <unknown>/50%  2        5        2         105s
backend   Deployment/backend   cpu: <unknown>/50%  2        5        2         2m
backend   Deployment/backend   cpu: <unknown>/50%  2        5        2         2m15s
backend   Deployment/backend   cpu: <unknown>/50%  2        5        2         2m30s
```

HPA: CPU 50% 사용량이 넘으면
(Min 2 → Max 5)

Cluster AutoScaler: Node 리소스 부족
시 EC2 추가



부하 테스트를 통한 Kubernetes 자동 스케일링 검증

Before: ReplicaSet 스케일링 이벤트 트리거
After: Pod 개수 증가 및 정상 배포 확인

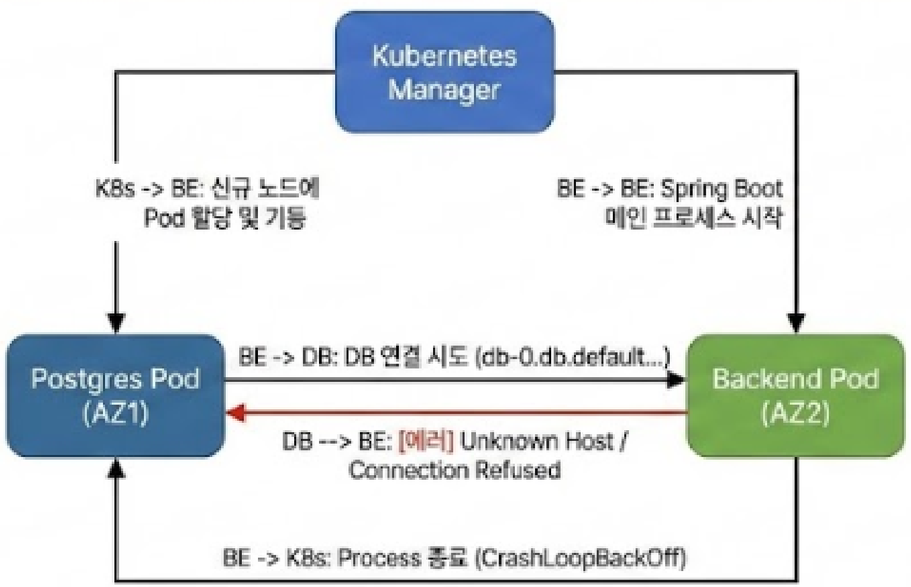
결과: 트래픽 증가 시 자동으로 Pod 확장하여 안정적 서비스 제공

EKS 설정 중 문제사항 - 1

문제1: AutoScaling 시 서비스 간 통신 불안정

배경 → 시스템을 EKS 환경에 배포 후 AutoScaling 적용한 뒤, 트래픽 증가로 새로운 Pod생성 시, 백엔드 Pod가 DB연결에 실패하여 지속적으로 CrashLoopBackOff 발생

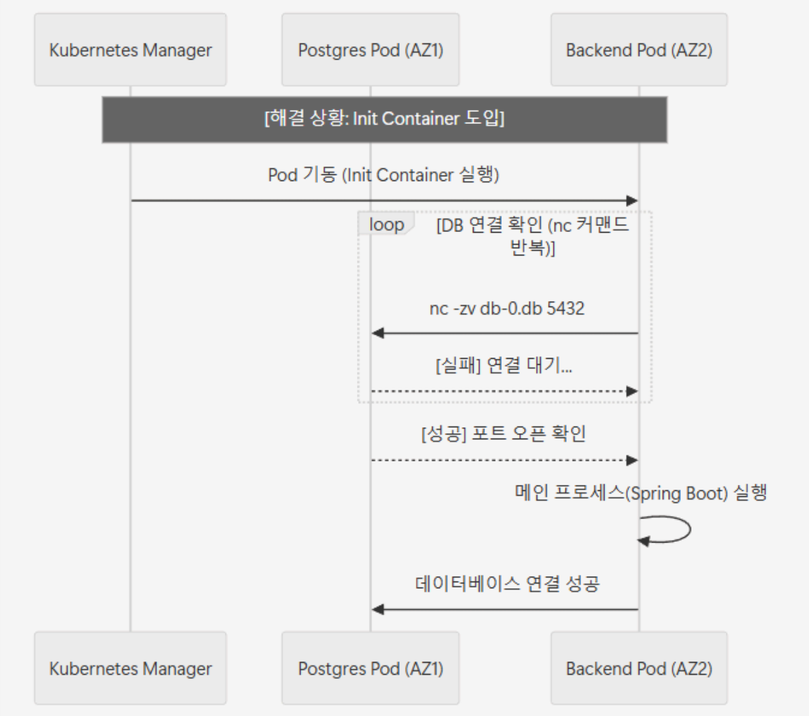
문제점 → 새로운 Pod생성 후 DNS 정보가 클라스터 전체에 전파되기 까지 시간이 필요한데, 백엔드가 DB의 DNS를 찾지 못하고 바로 시작을 수행하여 커넥션 에러 발생



해결방안

Init Container 도입 → 메인 컨테이너가 뜨기 전 busybox 이미지를 사용하여, DB: 5432포트가 열렸는지를 nc 명령어로 체크하는 로직 추가

DNS전파 대기 → 연결전까지 단순히 대기가 아닌, 연결 성공 시까지 재시도하도록 설계하여 인프라 전파 속도와 관계없이 안정적 동작



결과: Pod생성 시 발생하던 DB연결 오류가 모두 사라졌으며, AutoScaling과정에서 서비스 안정성 확보

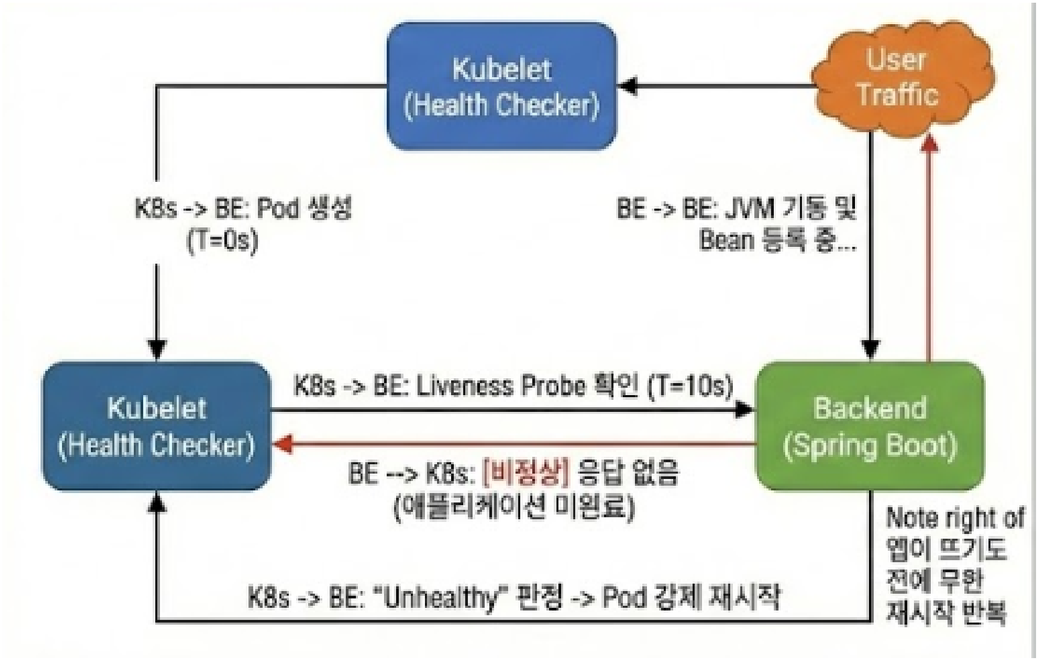
EKS 설정 중 문제사항 - 2

문제2: 무중단 배포 및 복구를 위한 가용성 최적화

배경 → Spring 백엔드와 Next.js 프론트를 롤링 업데이트로 배포하던 중, 새버전이 배포되자마자 기존 Pod가 삭제되면서 서비스 중단 현상이 발생

문제점 → Spring의 무거운 기동 시간으로 앱 동작에 대략 30~40s에 가까운 시간이 걸렸는데, k8s에서는 기본 Probe 설정이 앱이 뜨기도 전에 상태 Check를 수행하여 실패로 간주 재시작을 수행

부적절 포트 설정 → Liveness설정에 포트 정보 누락으로 상태 확인 자체가 불가능



해결방안

Probe설정 → 기존 InitialDelaySec를 10s → 45s로 수정하여 불필요한 재시작 방지

TCP Socket Check → 기존 Http요청으로 활성 여부 확인에서 Port활성 여부로 체크하도록 변경, 신뢰성 향상

배포 전략 수정 → maxUnavailable설정을 통하여 새로운 Pod가 완전히 준비되기 전까지는 기존 Pod를 죽이지 않도록 설정